

CHP: Data Structures and Algorithms

This document explains **how** the simulator represents the quantum state and how it implements gates and measurement. It maps directly to the data structures and functions in `chp.c`.

1. Representing a Stabilizer State

A stabilizer state is determined by (n) independent **stabilizer generators** $(\{g_1, \dots, g_n\})$ (Pauli products with phase (± 1) such that $(g_i|\psi\rangle = |\psi\rangle)$). CHP also keeps **destabilizer generators** $(\{\bar{g}_1, \dots, \bar{g}_n\})$ so that the full set $(\{g_i, \bar{g}_i\})$ forms a canonical basis for updating under gates (see the Aaronson–Gottesman paper).

1.1 Pauli as bits and phase

A single-qubit Pauli is one of I,X,Y,Z. On n qubits, a Pauli product is specified by:

- For each qubit j : two bits (x_j, z_j) :
 - $(0,0)=I, (1,0)=X, (1,1)=Y, (0,1)=Z$
- A **phase** $r \in 0,1,2,3$ meaning the overall factor is i^r (so $0 \rightarrow +1, 1 \rightarrow i, 2 \rightarrow -1, 3 \rightarrow -i$). For stabilizers we usually have $r \in 0,2 (\pm 1)$.

So one generator = n bits for X, n bits for Z, plus one phase. CHP packs these bits 32 per **unsigned long** for speed.

1.2 `struct QState` (from `chp.c`)

```
struct QState {
    long n;                // number of qubits
    unsigned long **x;      // (2n+1) × (packed) matrix of X bits
    unsigned long **z;      // (2n+1) × (packed) matrix of Z bits
    int *r;                // phase for each row: 0,1,2,3
    unsigned long pw[32];   // pw[i] = 2^i for bit indexing
    long over32;            // number of unsigned longs per row = (n>>5)+1
};
```

Rows 0 ... n−1: Destabilizer generators $\bar{g}_1, \dots, \bar{g}_n$ (the “X-bar” rows).

Rows n ... 2n−1: Stabilizer generators $g_1, \dots, g_n$ (the “Z-bar” rows).

Row 2n: Scratch row used for temporary computations (e.g. measurement, Gaussian elimination, printing).

So the “state” is the pair of tables: destabilizers (first n rows) and stabilizers (next n rows).

1.3 Initial state $0\dots 0$

For $0\dots 0$:

- Destabilizer row $i = X_i$ (X on qubit i , I elsewhere).
- Stabilizer row $i = Z_i$ (Z on qubit i , I elsewhere).

So initially: $x[i]$ has a 1 in column i only; $z[n+i]$ has a 1 in column i only; phases $r[i]=0$.

2. Gate Updates

The idea: for each gate U , every generator G is replaced by $U G U^\dagger$. So we update each row (each destabilizer and stabilizer) according to how that gate conjugates Pauli products.

2.1 CNOT (control b , target c)

Conjugation rules:

- $I_b \otimes A_c \rightarrow I_b \otimes A_c$
- $X_b \otimes I_c \rightarrow X_b \otimes X_c$
- $X_b \otimes X_c \rightarrow X_b \otimes I_c$
- $Z_b \otimes I_c \rightarrow Z_b \otimes I_c$
- $I_b \otimes Z_c \rightarrow Z_b \otimes Z_c$
- $Z_b \otimes Z_c \rightarrow I_b \otimes Z_c$

So in the table: for each row, if there is an X on the control, XOR the target's X into the row; if there is a Z on the target, XOR the control's Z into the row. The phase updates when both X on control and Z on target (and possibly both on the other qubit) due to $Y = iXZ$ and anti-commutation. The code does this with bit masks pwb , pwc and index arithmetic $b5 = b \gg 5$, $c5 = c \gg 5$ for the packed representation.

2.2 Hadamard on qubit b

H swaps X and Z on that qubit: $H X_b H^\dagger = Z_b$, $H Z_b H^\dagger = X_b$. So for each row, swap the X and Z bits in column b . If both X and Z were 1 (Y), the phase gets a factor -1 (r gets $r+2 \bmod 4$).

2.3 Phase (S) on qubit b

S : $X_b \rightarrow Y_b$, $Z_b \rightarrow Z_b$. So: if there is X on b , add Z on b (XOR Z bit); if both X and Z are set (Y), multiply by i (phase r gets $r+2 \bmod 4$).

3. Measurement (Z-basis)

Measuring qubit b corresponds to measuring the observable Z_b .

- **Commutation:** Check whether Z_b commutes with all stabilizer generators. Each generator is a product of Paulis; it commutes with Z_b iff on qubit b it is I or Z (no X or Y). So we look at the **X** part of the stabilizer rows at column b .

3.1 Determinate outcome

If every stabilizer generator commutes with Z_b , then Z_b is in the stabilizer (up to sign), so the state is an eigenstate of Z_b and the outcome is fixed (0 or 1). The code finds which destabilizer generator has X on b , then multiplies stabilizers to form the eigenvalue and returns 0 or 1 without randomness.

3.2 Random outcome

If some stabilizer generator has X (or Y) on b , it does not commute with Z_b ; the outcome is random 0 or 1. The algorithm:

1. Pick a pivot stabilizer row p that has X on b .
2. Replace the corresponding destabilizer row p by that stabilizer (so the new " $X\text{-bar}_p$ " is the old " $Z\text{-bar}_p$ ").
3. Set the stabilizer row p to Z_b (and phase 0 or 2 at random).
4. For every other row i (destabilizer or stabilizer) that has X on b , multiply that row by the new stabilizer row p so that the table remains canonical and the state is the post-measurement stabilizer state.

The random bit is the only place where "quantum randomness" appears; the rest is deterministic linear algebra over the Pauli table.

4. Gaussian Elimination and Printing the State

4.1 `gaussian(q)`

Puts the stabilizer generators into a **quasi-upper-triangular** form:

- First: a minimal set of generators that contain X or Y (in "upper" form). The number of these is returned: $g = \lceil \log_2(\text{number of nonzero basis states}) \rceil$.
- Then: generators that are Z -only (again in triangular form).

This is used to enumerate the basis states when printing the state in ket form.

4.2 `printstate(q)`

Prints the $2n$ generators: first n rows as destabilizers (with $\pm$ from phase), then a separator, then n stabilizers. Each generator is printed as a string of I, X, Y, Z over the n qubits.

4.3 `printket(q)`

Uses `gaussian(q)` and then a **seed** in the scratch row to iterate over all 2^g basis states and print them (e.g. $|00\dots 0\rangle, -|11\dots 0\rangle, \dots$). Only works if $g \leq 31$ (so at most 2^{31} basis states).

5. Row Operations

- **`rowcopy(q, i, k)`**: Set row i = row k (destabilizer/stabilizer and phase).
- **`rowswap(q, i, k)`**: Swap rows i and k (using scratch row).
- **`rowset(q, i, b)`**: Set row i to the b -th "canonical" Pauli: $b < n \rightarrow X_b$, $b \geq n \rightarrow Z_{b-n}$.
- **`rowmult(q, i, k)`**: Left-multiply row i by row k in the Pauli group (XOR bits, update phase via `clifford()`).

`clifford(q, i, k)` returns the phase (0–3) of the product (row i)-(row k) using the Pauli multiplication rules (e.g. $XY=iZ$, $XZ=-iY$, ...).

6. Program Structure: `struct QProg`

```
struct QProg {
    long n;           // number of qubits (inferred from circuit)
    long T;           // number of gates/instructions
    char *a;          // opcode per instruction: CNOT, HADAMARD, PHASE, MEASURE
    long *b;          // first qubit (control for CNOT, target for H/P/M)
```

```
    long *c;          // second qubit (target for CNOT only)
    int DISPQSTATE;    // print state (q/Q)
    int DISPTIME;
    int SILENT;
    int DISPPROG;
    int SUPPRESSM;
};
```

`runprog(h, q)` runs the circuit: for each instruction, it calls `cnot`, `hadamard`, `phase`, or `measure`, and optionally prints timing and state.

Next: [03_File_Format_and_Usage.md](#) — file format and command-line usage.